

Agent Task Contract: Week 5

Field	Value
Project	University Student-Support Case Agent (BSE4104)
Owner	Kabanda Arthur, Project / Requirements Lead
Date	30 September 2026
Version	1.0
Status	Week 5 contract; implementation integration is noted below

1. Purpose and Scope

This contract defines one bounded, auditable workflow for the student-support agent. It covers one student request at a time and does not grant the agent authority to make university decisions.

The Week 5 task is to help a student report an examination timetable clash, create a support ticket using the approved tool, and verify the newly created ticket's status before reporting the result. The agent must not imply that the clash has been resolved or that staff have approved an action.

2. Goal and Completion Criteria

For a sufficiently detailed examination-clash request, the agent will:

1. Use only details supplied by the student and any relevant retrieved evidence.
2. Create an EXAMINATION support ticket using `create_support_ticket`.
3. Use the returned ticket ID with `get_case_status` to verify the recorded status when the creation call succeeds and the round limit permits.
4. Report the ticket ID and the status returned by the tool, including when the status is `PENDING_STAFF_APPROVAL`.

The task is complete when the verified status is reported, or when the agent honestly explains why it could not complete the workflow and gives a safe next step. A pending staff review is a valid ticket outcome, not confirmation that the issue itself is resolved.

If required ticket information is missing or invalid, the agent asks the student to provide it and stops the current run. Cross-turn memory is not assumed; a later message is a new run unless a future, explicitly enabled session-memory feature provides otherwise.

3. Approved Capabilities

Capability	Purpose	Boundary
<code>create_support_ticket</code>	Record a student-support request.	Requires <code>student_id</code> , <code>category</code> , <code>summary</code> , <code>details</code> , and <code>urgency</code> . Examination or high-urgency tickets are recorded as <code>PENDING_STAFF_APPROVAL</code> .
<code>get_case_status</code>	Read an existing case or ticket status.	Requires a case/ticket ID. It does not change the record.
RAG retrieval	Supply source-grounded policy evidence where relevant.	Retrieval is performed by the application before the model response; it is not a callable function in the orchestrator's tool list. The agent must not invent policy when evidence is absent.

The function `allow-list` in `src/model_client.py` contains only the two tools listed above. An unknown tool name is not dispatched to a handler and produces an `UNKNOWN_TOOL` error result. The model may receive that result as context; the current loop does not implement an immediate-stop rule for this case.

4. Inputs and State

The workflow may use:

- ❖ The current student message and details explicitly provided in that message.
- ❖ Tool calls and tool results accumulated during the current orchestrator run.
- ❖ Retrieved evidence supplied by the application, including source identifiers where the retrieval result provides them.
- ❖ The current tool-round count.

The orchestrator keeps tool-protocol messages in memory during a run. It does not persist student-session history by default. `TurnResult.history` contains the assistant and tool messages appended during the run; it is not a complete, persistent audit record of the original request and retrieval sources. Traces must therefore record those separately and must redact student identifiers.

5. Limits and Runtime Behavior

- ❖ The default maximum is 5 tool-call rounds per run, configured by `MAX_TOOL_ROUNDS` and overridable when constructing `Orchestrator`.
- ❖ A round is one model response containing one or more tool calls. The limit is a limit on tool-call rounds, not a limit on all model requests.
- ❖ When the limit is reached and the model requests more tools, the orchestrator returns a `TOOL_ROUNDS_EXCEEDED` result for those calls, then makes one final model request with tools disabled. The final response must not claim that an unexecuted tool action succeeded.
- ❖ The current loop does not independently detect or block repeated identical tool calls. They consume the available round budget; the round cap bounds the loop.
- ❖ Tool validation and business-rule failures are returned as structured error results for the model to interpret. Recovery may continue within the round limit; the model must not fabricate a successful result.

6. Human Review and Safety Boundaries

The ticket tool records examination tickets with status `PENDING_STAFF_APPROVAL` and routes them for staff review. This is a staff approval state after ticket logging; it is not a separate student approval step before the tool call. The agent may report that a ticket is awaiting staff review but must not say that staff have approved it or that the requested change has been made.

The agent must not make or influence admissions, grading, disciplinary, or fee decisions. It must not create a ticket intended to alter those outcomes. It must not invent case status, ticket IDs, university policy, offices, or actions that a tool or retrieved source did not provide.

Use only sandbox/test records when capturing development traces. Do not include real student identifiers, credentials, or unnecessary personal details in committed evidence.

7. Stop and Recovery Conditions

The agent ends the current run when one of these conditions applies:

Condition	Required behavior
Ticket created and status verified	Report the returned ticket ID and verified status. If pending staff review, explicitly say it is awaiting staff review.
Required information is missing or rejected	Ask a concise clarification question and stop; do not invent missing fields.
Tool returns a structured error	Do not claim success. The model may explain the error or make a corrective call within the remaining round budget. If it cannot recover, give an honest next step.
Tool-round limit is reached	Use the final no-tools response to state that the task could not be completed in this run and give a safe next step.
Request is outside the agent's authority	Decline the decision/action and direct the student to an appropriate official channel when known from approved context.
No supporting policy evidence is available	Do not state an unsupported policy answer; explain the limitation or ask for clarification.

8. Verification and Evidence

1. A successful examination-clash trace showing ticket creation followed by a status lookup and a final response.
2. A failure/recovery trace, such as an invalid case ID or rejected ticket input, showing the error result and an honest recovery or stop.
3. A bounded-loop trace demonstrating the configured round limit and final no-tools response.

Each trace should include the sanitized student request, relevant evidence and source IDs if retrieval was used, tool names and sanitized inputs, returned results, round count, final response, and stop reason. Store traces under evidence/traces/ and retain no real student identifiers.

The existing focused test suite is `pytest tests/test_orchestrator.py -v`. Expected coverage includes multiple tool rounds, tool errors, and the round cap. Any additional contract guarantee must be covered by a test before it is reported as implemented.

9. Implementation Alignment Note.

The repository's `src/orchestrator.py` implements the tool loop and currently defaults to five tool-call rounds. However, `src/main.py` currently performs RAG retrieval and calls the model directly; it does not route the `/chat` request through Orchestrator. Therefore, this contract describes the Week 5 bounded workflow target, but the end-to-end application path is not yet integrated with that loop. The team must either wire the target workflow through Orchestrator and verify it, or clearly label traces as orchestrator tests rather than claiming the current `/chat` endpoint runs the bounded loop.

The repository currently contains Prompt Specifications v1.0-v3.0, with v3.0 as the default. This contract does not assume a v4.0 prompt exists. Prompt version changes and any changes to loop limits or stop behavior must be reflected in the implementation, tests, and this document.